Design and implementation of a traffic light with a day and night cycle using Arduino

Misael Gallardo
Universidad de Atacama
Ingeniería Civil Informática
Copiapó, Chile
misael.gallardo.19@alumnos.uda
.cl

***Abstract*—This project focuses on the design and implementation of a synchronous traffic light system that can change with a pulse signal, having two upper states and four lower or common states, using an Arduino microcontroller and a button as the switching devices. Modeled as a Mealy finite state machine, the system allows users to input the change signal, causing the system to momentarily alter its synchronous sequence. If the change signal is detected in an incorrect state, it is ignored.**

The main objective of the project is to provide a working example of a synchronous system that interacts with hardware input and provides visual feedback via LEDs. Using Arduino code, the system processes the button input and controls the LED outputs. A thorough approach to documentation and modeling is adopted to ensure that each step of development is clearly defined and well understood, with the aim of reinforcing knowledge of automata and finite state machine theory. Therefore, this project serves not only as a practical application of sequential systems but also as an educational tool for exploring the design and optimization of sequential logic systems. By applying a Mealy machine model, the system was optimized to reduce state complexity while maintaining proper functionality, thus demonstrating the benefits of finite state machine theory in real-world applications.

Keywords: Arduino, synchronous systems, upper state, lower or common state.

Falta sección de introduccion

I. Metodología

Para asegurar el correcto funcionamiento del sistema, se identificaron los siguientes pasos clave para asegurar el correcto funcionamiento del sistema y permitir que su desarrollo fluyera sin mayores percances:

1) **Definición de requisitos:** En esta etapa, se estableció la necesidad de utilizar un botón como señal de cambio para un sistema establecido sincronico.
2) **Diseño del sistema:** Se diseñó el sistema en torno a una lógica secuencial, modelada y optimizada como un autómata de Mealy.
3) **Desarrollo del código en Arduino:** Se escribió el código para controlar el flujo de entrada, verificar la secuencia, y proporcionar retroalimentación visual mediante LEDs.
4) **Validación y pruebas:** La realización de pruebas físicas para validar el correcto funcionamiento del sistema es vital, asegurando que el sistema responda apropiadamente. Además, se verifica que el tiempo de activación y el restablecimiento automático funcionaran de acuerdo a lo previsto.

Los principales componentes del sistema son tres:

1) **Microcontrolador Arduino Uno**, que actúa como la unidad de procesamiento principal, gestionando las entradas y salidas del sistema.
2) **Un botones (“A”)**, utilizados para ingresar la señal de cambio del sistema
3) **Diodos emisores de luz (LEDs)**, que sirven como dispositivos visuales de avance, alerta y detención como sería en un sistema real.

Estos elementos, en conjunto, son los bloques fundamentales con los cuales se puede construir el sistema.

II. Diseño del Sistema

El sistema está diseñado en torno a una lógica secuencial, donde las luces LEDs siguen una secuencia síncrona hasta recibir una señal de cambio. Se trata de un sistema de señales visuales para poder pasar una vía transitada, y lograr reducir la espera de peatones. Este sistema tiene 2 super estados “Dia”

(“S_{0X}“) y “Noche” (“S_{1X}.”) Para permitir un mejor entendimiento del sistema a desarrollar, es posible moldearlo como un autómata de estados finitos, particularmente como un autómata de Mealy, ya que las salidas dependen directamente de las transiciones entre estados y las entradas actuales, lo cual permite una respuesta más rápida y eficiente[1].

En este caso, el botón (“A”) dará una señal de cambio si el sistema está en un estado en concreto (**S_{X0}**), la cual dará una advertencia antes de este.. Cualquier otro estado que reciba la señal será ignorado por tema de seguridad vial, donde el sistema continuará hasta reiniciarse y volver esperar el estado correcto. Por el contrario, si la señal es ingresada correctamente, el sistema activa una señal interrupdida de alta y baja energía (representada por el encendido del LED amarillo 1 [“2A”]) durante 1 segundos, indicando que el cambio de sistema fue correctamente ingresada. Con esta configuración, se identifican en primera instancia 7 estados principales, cada uno con transiciones bien definidas para la entrada de 0 o 1:

1) **Estado inicial (S_{X0}):** El sistema espera el input del botón “A”. Si no es presionado el sistema continuará secuencialmente hasta reiniciarse. Inicia con LED verde 1 y LED rojo 2 por 1 segundo.
2) **Estado intermedio 1 (S_{X1}):** .Si el estado anterio recibió la señal de cambio, el sistema mantendra el LED verde 1 y comenzara a encender intermitentemente el LED amarillo 1, por 1 segundo
3) **Estado intermedio Bajo 1 (S_{X2}):** Si **S_{X0}** no recibio la señal entonces apagar el LED verde 1 y encendera el LED amarillo 1 por 1 segundo. desde este punto la señal de cambio será ignorada debido a que no hay tiempo suficiente para avisar de un cambio del sistema.
4) **Estado intermedio Bajo 2 (S_{X3}):** Se Apagara el LED amarillo 1 y se encendera el LED rojo 1 por 1 segundo.
5) **Estado intermedio 2 (S_{X4}):** Este estado es la unión de los estados Alto y Bajo. Los LEDs anteriores (LED rojo 1 en caso de estado bajo y LED verde 1 con amarillo 1), se pagaran para encender el LED verde 2. Se entendió que el cambio de estado es de un solo sentido por lo que el botón “A” sigue siendo ignorado
6) **Estado Intermedio 3 (S_{X5}):** Se apaga el LED verde 2 y se enciende el LED amraillo 2.
7) **Estado Intermedio 4 (S_{X6}):** Se apgara el LED amarillo 2 y se encendera el LED rojo 2 y LED verde 1
8) Se reinicia el sistema

Estos 8 estados se le puede denominar como estados comunes o normales, debido que son repetitivo en los 2 super estados siguiente:

1) **DIA:** El sistema de semáforo funciona de manera normal y sincronizada, encendiendo un LED a la vez.
2) **NOCHE:** El sistema de semáforo cambia para siempre tener un LED de alerta (Amraillo) si la vía es utilizable.

Cada uno de estos estados está estrictamente definido, lo que permite un control preciso del flujo del sistema. Este diseño garantiza que cualquier estado que reciba la señal que no sea el **S_{X0}** no produzca ninguna interferencia.

La figura 1 presenta cada secuencia definida dentro del autómata de Mealy, donde la entrada del botón (“A”) y la salida del sistema (“Z”) son representados en el formato “A/Z”. Si se presiona “A” mostrará como entrada “1/”, y dará como resultado una salida “1” si pertenece a la entrada **S_{X0}** y “0” si el estado es erróneo, provocando que sea ignorado. En caso que el botón no sea presionado “0/” la salida será “0”, representando al sistema en un estado de espera de una entrada válida. En la tabla 1 puede apreciarse cómo las entradas dado un estado, da lugar al siguiente estado dada las entradas estándar (0, 1).

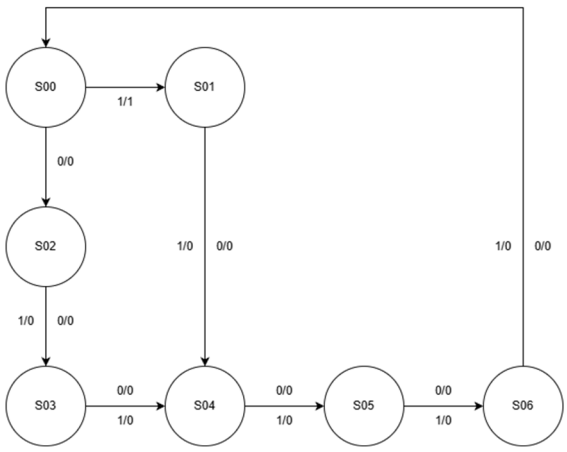

Fig. 1. Diagrama no optimizado de una Máquina de Mealy para 2 semáforos vehiculares y uno peatonal de super estado 0 (“DIA"). Sin optimizar

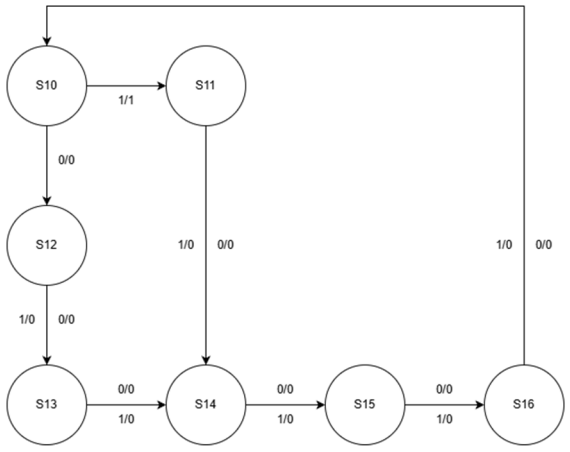

Fig. 2. Diagrama no optimizado de una Máquina de Mealy para 2 semáforos vehiculares y uno peatonal de super estado 1 (“NOCHE”). Sin optimizar

TABLA I. Tabla de Estados Finitos.

Estado Actual	Entrada A	Estado siguiente	Salida
S0	0	S2	0
S0	1	S1	1
S1	0	S4	0
S1	1	S4	0
S2	0	S3	0
S2	1	S3	0
S3	0	S4	0
S3	1	S4	0
S4	0	S5	0
S4	1	S5	0
S5	0	S6	0
S5	1	S6	0
S6	0	S0	0
S6	1	S0	0

Para el super estado 1 la tabla no sufre cambios..

El autómata de Mealy es optimizable al observar que el estado S_{06} es el reinicio del sistema por lo que se puede reemplazar por S_{00}.

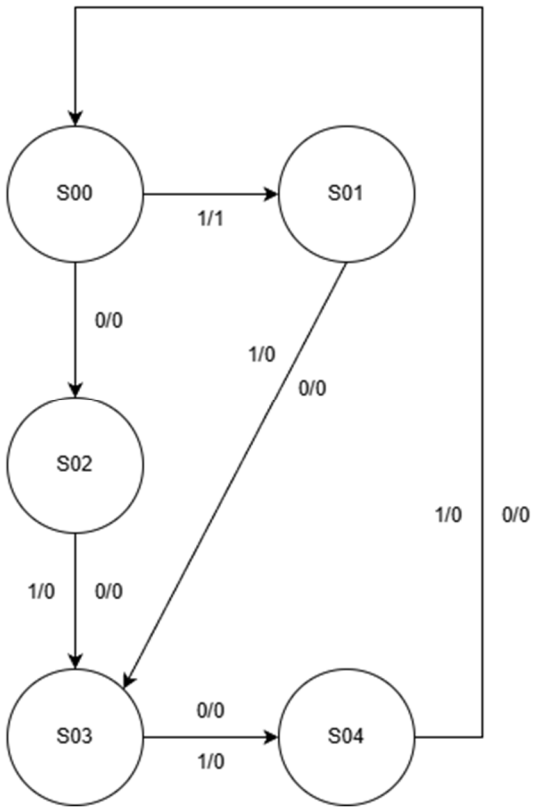

Fig. 3. Nuevo autómata de Mealy, el estado S_{06} y S_{05} ha sido eliminado, y su reinicio es más rápido. Super estado 0 (“DIA”)

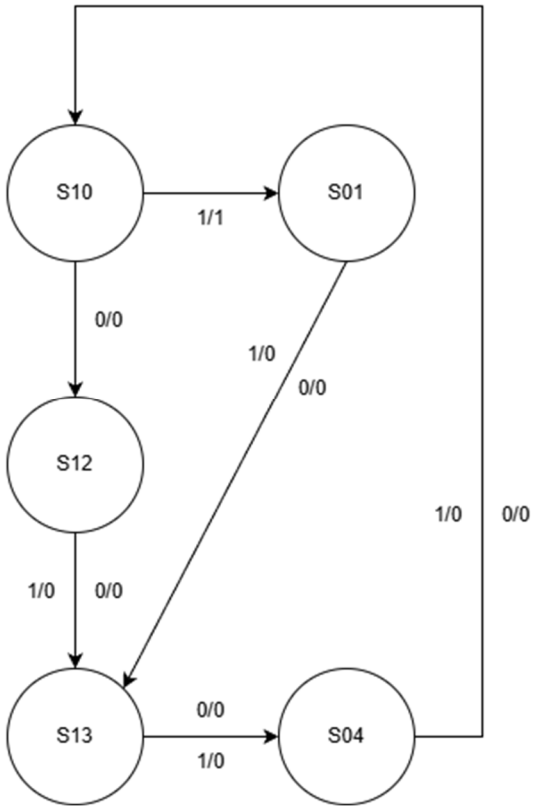

Fig. 4. Nuevo autómata de Mealy, el estado S_{06} y S_{05} ha sido eliminado, y su reinicio es más rápido. Super estado 1 (“NOCHE”)

Al optimizar el diagrama, se ha generado un sistema más rápido y eficiente al no repetir un proceso final igual que el inicial.

III. Implementación

La implementación técnica de este proyecto se centró tanto en el desarrollo del código como en la integración de los componentes físicos, manteniendo la simplicidad del hardware gracias a la optimización de la lógica mediante el autómata de Mealy de la figura 3 y 4. La eliminación de los estados innecesarios permite una implementación más eficiente.

El microcontrolador Arduino Uno fue programado para manejar las entradas de un botón y procesarlo como una señal de cambio. La secuencia se dividió en las siguientes fases:

1) **Inicialización del sistema:** Se definen el estado inicial la variable necesaria para el botón (A) y los LEDs utilizados para el sistema. Se configuran los pines de entrada y salida del microcontrolador, y se inicializan los contadores y variables necesarias para gestionar la secuencia.
2) **Lectura de entrada:** El usuario presiona el botón, el Arduino registra la entrada y compara el estado actual con el correcto para el cambio(S_{X0}) En esta etapa, el código está estructurado de forma que cada estado se gestiona mediante el autómata, procesando la secuencia según los valores de entrada.
3) **Comparación de la estado y activación:** Si se recibe la señal de cambio, el sistema compara el valor ingresado con el estado correcto. Si es incorrecto, el sistema continúa normalmente..
4) **Cambio aceptado:** Si se ingresa en el estado correcto, el Arduino envía una señal de activación que enciende un LED intermitentemente por 1 segundos, indicando que el sistema secuencial va a cambiar.
5) **Restablecimiento automático:** Después del último estado, el sistema se reinicia automáticamente, listo para recibir una nueva señal de cambio. Este reinicio incluye el restablecimiento del sistema secuencial inicializando los LEDs desde el estado S_{X0}.
6) **Estado del día:** Este estado es el modo que el sistema menaje de manera autónoma el ciclo de dia y noche, donde cambia el modo de encendido de los LEDs

Con ello, el sistema debería ser capaz de manejar todos los estados disponibles del sistema, impidiendo estados no válidos y permitiendo una interacción con el usuario fluida, donde se comunica de manera efectiva los estados y funcionamiento de la máquina.

IV. Coste de Producción

Dado que parte del objetivo de este proyecto, es producir físicamente el sistema, se detallan los costos de los componentes utilizados para el desarrollo, expresados en pesos chilenos:

TABLA II. Precios de los Componentes

N°	Componentes necesarios para producir			
	Componente	**Cantidad**	**Coste Unitario**	**Costo Total**
1	Arduino Uno	1	$16.990	$16.990
2	Líneas Dupont	21	$250	$5.250
3	LEDs colores	11	$50	$550
4	Resistencia 1KΩ	9	$380	$3.420
5	Botones de 4 pines	1	$290	$290
Total				$26.500

La tabla 2 que se presenta muestra los costos de los componentes necesarios para el sistema. Sin embargo, es importante señalar que los precios que se indican se obtuvieron de mercados chinos o bazares electrónicos en los que se utilizaron modelos no oficiales, como unidades Arduino Uno sin marca. Estos precios tienden a ser más bajos que los de los modelos oficiales y no tienen en cuenta los gastos adicionales, como la mano de obra para el montaje, los derechos de importación y los costes de envío. Esto podría incurrir en un coste total considerablemente mayor.

Este análisis sirve como estimación del coste mínimo en circunstancias ideales, en lugar de ser un reflejo exhaustivo del coste real de fabricación e implementación del sistema.

V. Resultados

El proyecto consiguió el resultado deseado, cumpliendo con todos los requisitos establecidos en un plazo óptimo. La demostración práctica y la implementación se completaron con éxito y el sistema funcionó como se esperaba. El semáforo funciona correctamente, aun después de cambiar la secuencia síncrona con el botón “A” brindando información

visual a través de LED amraillo 1 del futuro cambio de esta secuencia y permitiendo el tiempo necesario para reaccionar a este y evitar accidentes no deseados en un modelo real.

Sin embargo, un problema notable emergió durante la etapa de pruebas. Dado el uso de funciones de espera para controlar el parpadeo de los LEDs, el sistema ocasionalmente enciende LED que por arquitectura no debería encender, o parpadeo irregular de un LED específico en momento incorrecto. Estos problemas ocurren en el momento de cambiar de super estados. Otro desafío fue la validación del botón debido que este debe mantenerse presionado en el estado S_{X3} hasta el inicio del reinicio del estado S_{X0}. Esta limitación deja la ventana abierta para mejorar y optimizar el diseño actual del sistema.

A pesar de este inconveniente, el comportamiento del sistema bajo condiciones normales es estable y confiable. El problema previamente mencionado otorga una valiosa perspectiva sobre el manejo del tiempo sobre el sistema. Eso remarca la necesidad de, en futuras iteraciones, diseñar e implementar sistemas de tiempo más precisa, posiblemente haciendo uso de mecanismos basados en sensores de luz o una herramienta externa de tiempo, de tal manera que se asegure una respuesta correcta a la estructura sincrónica y predecible, independiente del super estado o cambio de estado común.

VI. Conclusión

En conclusión, mientras sistema funciona como se esperaba bajo un uso típico, la limitación observada sirve como una experiencia importante. Enfatiza la importancia de arquitectura y control de los tiempos del sistema que tienen como propósito retroalimentar a los usuarios.

El proyecto del semáforo a pesar de que se terminó de una manera rápida durante el taller presencial, el análisis fue algo que estuvo trabajando durante días; El código acabó siendo lo más complicado de lo esperado, debido a que al principio se trabajo con sentencias comparadas para tener un esquema básico de la función de los super estados por separados, para luego cambiar estas funciones por estados reales..

Aun así, este fue un proyecto divertido, un desarrollo didáctico que permite comprender mucho más allá el funcionamiento de los sistemas y aprender que el Arduino funciona de una manera concreta, y si no es así pues o no funciona, o crea falsos positivos dolorosos de corregir.

VII. Referencias

[1] M. Melendez, “ARQUITECTURA DE SISTEMAS DIGITALES DE CONTROL: Un Enfoque Práctico”, pp. 6-11, no publicado

seria bueno que hubieras incluido figuras (imágenes) de como quedo implementado (físico y en simulador), así como los correspondientes enlaces a los vídeos y simulaciones.

Buen trabajo al equilibrar las columnas de la ultima pagina